

Repository Summary — Code Sample

Yufeng (Alex) Xia · yufeng.xia@ed.ac.uk · University of Edinburgh

PRIMARY REPOSITORY: MORPHLING

- **URL** <https://github.com/netsys-edinburgh/morphling>
- **What it is** An emulator for distributed machine-learning training on heterogeneous edge devices. It runs *unmodified* training scripts against a unified backend — real or emulated — by intercepting backend dispatch calls, fitting performance models from real measurements, decoupling memory demand from device count, and driving execution on event-driven virtual time. This makes large-scale what-if studies possible without owning a physical device fleet.
- **Status** Research artifact of *Morphling: Emulator for Distributed Machine Learning at the Edge*, EdgeSys’26 (co-located with ACM MobiSys 2026), **Best Paper Award**. DOI 10.1145/3812836.3814779.
- **Scale** ~18,000 lines of C++/CUDA under `csrc/` plus a Python surface; CMake build; CI covering build, formatting, CodeQL, protobuf breaking-change and numerical-consistency checks.
- **My share** **91 commits** out of 519 in the repository, merged through reviewed pull requests. I was responsible for one clearly delimited subsystem: **the inter-device transport data plane and its performance**.

What I built

- **Event-driven transport** (`csrc/backend/proxy_svr.cc`, `proxy_cli.cc`, `server_base.cpp`). The proxy server and client that carry tensor traffic between emulated devices, built on libevent: non-blocking connection handling, event-loop threads, batched send/receive accounting, and connection lifecycle management. Emulation fidelity is bounded by this path — if a transfer costs more than the device being emulated would have spent, the emulated timeline is wrong — so the work was fundamentally about driving transfer cost down and making it predictable.
- **Zero-copy and pinned-memory path**. A `CudaPinnedMemoryPool` that allocates `cudaHostAlloc` buffers in power-of-two buckets and recycles them, removing per-transfer allocation from the hot path; zero-copy scatter-gather buffers (`evbuffer_add_reference` with `shared_ptr` cleanup) so that tensor payloads are transmitted without staging copies; CUDA host registration and worker-thread pinning, each introduced only after a dedicated microbenchmark showed it paid for itself (`tests/cpp/test_cuda_host_register_tensor.cpp`).
- **Serialisation cost study**. Instrumented the full send path (serialise / compute size / transmit) and compared Protocol Buffers against FlatBuffers under realistic message sizes, then chose the framing and batching strategy from the measurements rather than from folklore. The finding that FlatBuffers is *slower* in the Python path — contrary to the usual assumption — is documented in the repository.
- **Distributed virtual-clock synchronisation**. Virtual time must stay consistent across emulated devices or the emulator silently produces wrong results. I implemented the synchronisation mechanism and compared alternative strategies (`csrc/base/virtual_clock.cc`, `scripts/sync_virtual_time.py`, `scripts/compare_sync_strategies.py`).
- **Failure-mode debugging at scale**. Diagnosed and fixed deadlocks that appeared only under device failure and connection teardown, and the device-scale-zero condition — the class of bug that does not reproduce on two nodes.
- **Measurement tooling**. Per-device timeline and Gantt visualisation of the emulated execution (`scripts/timeline_plot.py`, `scripts/run_mixed_devices.py`), used both for debugging and for figures in the paper.

Where to look

The following files carry the bulk of my contribution and are the ones I would point a reviewer to:

File	Role	My churn
<code>csrc/backend/proxy_svr.cc</code>	Event-driven proxy server, batching, scheduling hooks	+964 / -269
<code>csrc/backend/server_base.cpp</code>	Connection layer, send/receive accounting	+743 / -228
<code>csrc/backend/proxy_cli.cc</code>	Client transport, <code>CudaPinnedMemoryPool</code>	+510 / -156
<code>csrc/backend/device_tracker.cc</code>	Distributed device state tracking	+387 / -48
<code>csrc/base/virtual_clock.cc</code>	Virtual-time synchronisation	—
<code>tests/cpp/test_cuda_host_register_tensor.cpp</code>	Pinned-memory microbenchmark	+407 / -89

My commits can be listed directly with `git log --author="Yufeng Xia" --author="xyf2002"`. Pull requests merged from my branches include `benchmark`, `throughput-optimization`, `fix-device-scale-zero` and `jyang-opt`.

Documentation I wrote for this repository: analyses of serialisation performance, virtual-time synchronisation strategies, connection cleanup, device-failure deadlocks and message-size behaviour. I treat a non-obvious performance result as unfinished until it is written down, and these documents are the record of that.

SECONDARY REPOSITORY: TENSOROPS (SOLE AUTHOR)

- **URL** <https://github.com/xyf2002/tensor-compiler>
- **What it is** A multi-target MLIR tensor compiler with operator fusion, written entirely by me. It defines a **TensorOps** dialect (`ten.matmul`, `ten.relu`, `ten.add`), fuses `matmul+relu` at the dialect level, and lowers through `Linalg` either to LLVM for CPU JIT execution or to NVVM/ROCDL, ending in PTX and cubin launched through the CUDA Driver API.
- **Why included** Morphling shows how I work inside a large shared codebase under review; TensorOps shows what I produce alone, end to end — dialect design, pass pipeline, GPU backend, benchmark harness and documentation — which is the ability the iSING Lab call asks for.

Further code, including the LLVM auto-quantization work behind my first-author manuscript *CoSenseAQ*, is available on request.